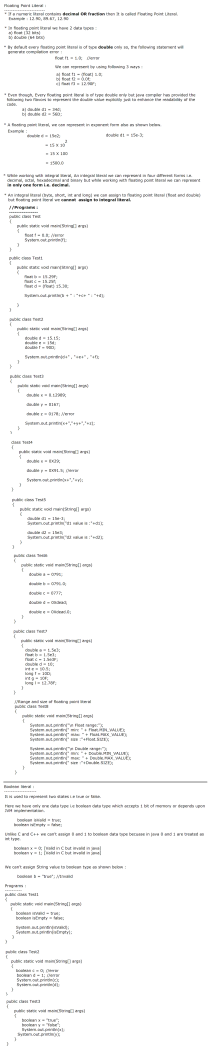

Floating Point Literal :

- If a numeric literal contains **decimal OR fraction** then It is called Floating Point Literal.
  Example : 12.90, 89.67, 12.90

- In floating point literal we have 2 data types :
  a) float (32 bits)
  b) double (64 bits)

- By default every floating point literal is of type **double** only so, the following statement will generate compilation error :

```
float f1 = 1.0;   //error
```

We can represent by using following 3 ways :

```
a) float f1 = (float) 1.0;
b) float f2 = 0.0f;
c) float f3 = 12.90F;
```

- Even though, Every floating point literal is of type double only but java compiler has provided the following two flavors to represent the double value explicitly just to enhance the readability of the code.

```
a) double d1 = 34d;
b) double d2 = 56D;
```

- A floating point literal, we can represent in exponent form also as shown below.
  Example :

```
double d = 15e2;                    double d1 = 15e-3;
```

= 15 X 10^2

= 15 X 100

= 1500.0

- While working with integral literal, An integral literal we can represent in four different forms i.e. decimal, octal, hexadecimal and binary but while working with floating point literal we can represent **in only one form i.e. decimal.**

- An integral literal (byte, short, int and long) we can assign to floating point literal (float and double) but floating point literal we **cannot assign to integral literal.**

//Programs :

```
----------------
public class Test
{
    public static void main(String[] args)
    {
        float f = 0.0; //error
        System.out.println(f);
    }
}

public class Test1
{
    public static void main(String[] args)
    {
        float b = 15.29F;
        float c = 15.25f;
        float d = (float) 15.30;

        System.out.println(b + " : "+c+ " : "+d);

    }
}

public class Test2
{
    public static void main(String[] args)
    {
        double d = 15.15;
        double e = 15d;
        double f = 90D;

        System.out.println(d+" , "+e+" , "+f);
    }
}

public class Test3
{
    public static void main(String[] args)
    {
         double x = 0.12989;

         double y = 0167;

         double z = 0178; //error

        System.out.println(x+","+y+","+z);
    }
}

 class Test4
 {
     public static void main(String[] args)
     {
         double x = 0X29;

         double y = 0X91.5; //error

         System.out.println(x+","+y);
     }
 }

 public class Test5
 {
     public static void main(String[] args)
     {
         double d1 = 15e-3;
         System.out.println("d1 value is :"+d1);

         double d2 = 15e3;
         System.out.println("d2 value is :"+d2);
     }
 }

  public class Test6
  {
      public static void main(String[] args)
      {
          double a = 0791;

          double b = 0791.0;

          double c = 0777;

          double d = 0Xdead;

          double e = 0Xdead.0;
      }
  }

  public class Test7
  {
      public static void main(String[] args)
      {
        double a = 1.5e3;
        float b = 1.5e3;
        float c = 1.5e3F;
        double d = 10;
        int e = 10.5;
        long f = 10D;
        int g = 10F;
        long l = 12.78F;
      }
  }

   //Range and size of floating point literal
   public class Test8
   {
       public static void main(String[] args)
       {
           System.out.println("\n Float range:");
           System.out.println(" min: " + Float.MIN_VALUE);
           System.out.println(" max: " + Float.MAX_VALUE);
           System.out.println(" size :"+Float.SIZE);

           System.out.println("\n Double range:");
           System.out.println(" min: " + Double.MIN_VALUE);
           System.out.println(" max: " + Double.MAX_VALUE);
           System.out.println(" size :"+Double.SIZE);
       }
   }
```

Boolean literal :

It is used to represent two states i.e true or false.

Here we have only one data type i.e boolean data type which accepts 1 bit of memory or depends upon JVM implementation.

```
    boolean isValid = true;
  boolean isEmpty = false;
```

Unlike C and C++ we can't assign 0 and 1 to boolean data type becuase in java 0 and 1 are treated as int type.

```
boolean x = 0; [Valid in C but invalid in java]
boolean y = 1; [Valid in C but invalid in java]
```

We can't assign String value to boolean type as shown below :

```
boolean b = "true"; //Invalid
```

Programs :

```
----------
public class Test1
{
   public static void main(String[] args)
   {
      boolean isValid = true;
      boolean isEmpty = false;

      System.out.println(isValid);
      System.out.println(isEmpty);
    }
}

 public class Test2
 {
    public static void main(String[] args)
    {
      boolean c = 0; //error
       boolean d = 1; //error
       System.out.println(c);
       System.out.println(d);
    }
 }

  public class Test3
  {
      public static void main(String[] args)
      {
          boolean x = "true";
          boolean y = "false";
          System.out.println(x);
        System.out.println(y);
      }
  }
```